Compte rendu – Ticket'Tournelle, portail de gestion des incidents

Atelier de Professionnalisation · BTS SIO SLAM 2e année · Lycée La Tournelle

KANSIETOKO Yuung Addi

Année 2026–2027 · Debian 13, Apache, PHP 8.4, MariaDB · État au 5 novembre 2026

1. Contexte et problématique

Le Lycée La Tournelle, à La Garenne-Colombes, regroupe plus de 200 personnels. Aujourd'hui, les incidents techniques (vidéoprojecteur en panne, accès ENT impossible, imprimante bloquée) sont signalés par des canaux éparpillés : fichier Excel partagé, SMS, notes vocales WhatsApp, appels directs aux référents numériques. Résultat : des demandes perdues, des retards de traitement, aucun chiffre pour la direction et une surcharge pour les référents.

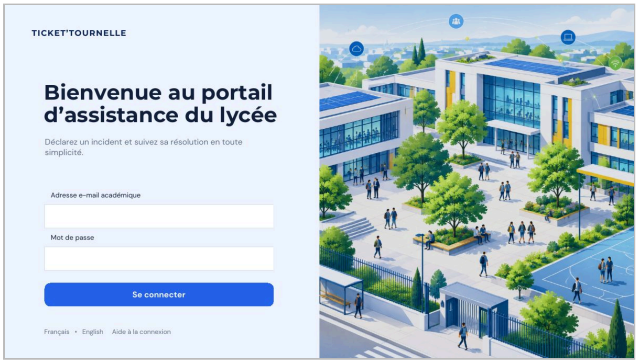
Le projet d'Atelier de Professionnalisation de BTS SIO 2 consiste à réaliser Ticket'Tournelle, un portail web où chaque personnel déclare un incident et suit sa résolution, où les techniciens traitent les tickets, et où la direction consulte des indicateurs. La maîtrise d'ouvrage impose 9 contraintes, notamment : un compte nominatif par personne, une interface en français et en anglais, un serveur interne installé et administré par nous (pas d'hébergeur clé en main), et une gestion des comptes administrateurs conforme aux recommandations de l'ANSSI.

2. Organisation

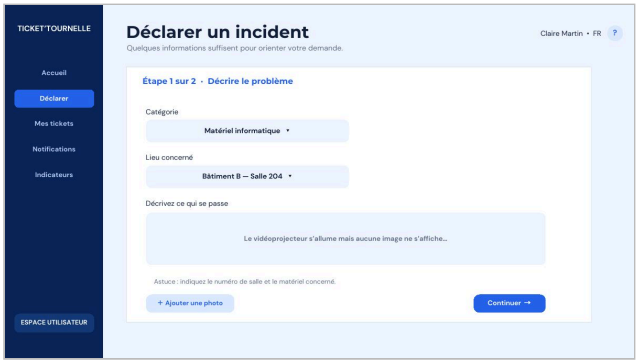
Le projet est piloté sur un tableau Trello organisé en 10 phases : cahier des charges, pilotage, serveur web interne, base de données, développement, sécurité, tests, documentation, mise en production et restitution. Chaque carte a sa checklist et des étiquettes (contrainte de la maîtrise d'ouvrage, sécurité, livrable). Le code est versionné dans un dépôt GitHub privé.

3. Phase 1 : le cahier des charges et les maquettes

J'ai analysé et reformulé les besoins, rédigé le cahier des charges, les user stories, le cycle de vie d'un ticket (ouvert, pris en charge, en attente, résolu, clos), les catégories et priorités, puis j'ai réalisé les maquettes des écrans principaux pour les trois rôles : utilisateur, technicien et administrateur.



Connexion, avec choix de la langue



Déclaration d'un incident en deux étapes



File de traitement du technicien



Indicateurs pour la direction

4. Phase 4 : la base de données

J'ai conçu le MCD et le MLD à partir des entités et des règles de gestion du cahier des charges. La base MariaDB compte 13 tables, avec leurs contraintes et leurs index, et 5 vues SQL qui préparent les indicateurs du tableau de bord (statistiques par catégorie et par priorité, temps de résolution, salles les plus touchées). J'ai ajouté quatre tables au modèle de départ, chacune pour un besoin écrit dans le cahier des charges :

- `salle` : pour localiser les incidents et repérer les salles où ils reviennent ;
- `transition_statut` : le cycle de vie est stocké dans la base, et un changement de statut non prévu est refusé ;
- `reinitialisation_mdp` : pour un lien de réinitialisation du mot de passe à usage unique ;
- `journal_securite` : pour tracer les connexions et les actions d'administration.

Les libellés des tables de référence existent en français et en anglais, pour afficher l'interface dans la langue de chaque utilisateur. Les mots de passe ne sont jamais stockés, seulement leur empreinte Argon2id. Les comptes ne sont pas supprimés mais désactivés, pour que l'historique des tickets reste lisible. L'application se connecte avec un compte SQL dédié qui n'a que les droits de lecture et d'écriture sur les données (principe du moindre privilège). Un jeu de test fournit 8 comptes fictifs et 20 tickets ; il sera retiré avant la mise en production.

5. Phase 3 : le serveur web interne

Conformément à la contrainte d'un serveur administré par nous, j'ai installé une machine virtuelle Debian 13 sans interface graphique, nommée `srv-ticket.tournelle.lan` (2 processeurs, 2 Go de mémoire). Le compte root est désactivé : l'administration passe par un compte nominatif membre du groupe sudo, pour que chaque action soit rattachée à une personne, comme le recommande l'ANSSI.

J'ai automatisé toute l'installation dans un script, `provision.sh`, qu'on peut relancer sans risque. Il installe et configure :

- Apache 2.4 et PHP 8.4, avec MariaDB qui n'écoute qu'en local, jamais sur le réseau ;
- HTTPS avec un certificat, TLS 1.2 et 1.3 uniquement, et la redirection de HTTP vers HTTPS ;
- les en-têtes de sécurité HTTP, et un seul dossier public exposé par Apache : le code et la configuration restent hors de la racine web ;

- le pare-feu UFW (seuls les ports 22, 80 et 443 sont ouverts, avec limitation des tentatives sur SSH) et Fail2ban ;
- la connexion SSH par clé, avec root interdit et la connexion par mot de passe coupée dès qu'une clé est déposée, et les mises à jour de sécurité automatiques ;
- une sauvegarde automatique de la base, déclenchée chaque nuit à 2 h 30 par un minuteur systemd.

Le mot de passe du compte SQL de l'application est généré sur le serveur et rangé dans un fichier de configuration protégé, hors de Git. Deux scripts de tests vérifient ensuite la base et la sécurité du serveur. J'ai vérifié depuis mon PC que le site répond en HTTPS, que TLS 1.1 est refusé, que les en-têtes de sécurité sont présents et que le code source n'est pas accessible.

6. Avancement et suite

Phase	État
1. Cahier des charges, user stories, maquettes, MCD et MLD	Terminé, validation écrite de la maîtrise d'ouvrage à obtenir
2. Pilotage (Trello, dépôt Git)	En cours
3. Serveur web interne (Debian, Apache, PHP, MariaDB, HTTPS)	Installé et durci, l'envoi des e-mails reste à configurer
4. Base de données, jeu de test, compte à privilèges minimaux	Terminé
5 à 10. Développement, sécurité applicative, tests, documentation, production, restitution	À venir

La prochaine étape est le développement de l'application : authentification et comptes nominatifs, rôles et droits, déclaration et suivi des tickets, espace technicien, notifications en français et en anglais, puis le tableau de bord. Ce projet me fait travailler à la fois l'administration système, la base de données et la sécurité, sur un vrai besoin d'établissement.